# The Transformer

## Attention Is All You Need

Lebnet Tech Fellows Program

# Roadmap

The Problem: Sequence Modelling

Embedding and Position

Attention

Multi-Head Attention

The Transformer Block

Output Head

Training and Generation

The Big Picture

# Language Modelling

Input: a sequence of discrete tokens $w_1, w_2, \ldots, w_T$ from a vocabulary $\mathcal{V} = \{1, \ldots, V\}$.

By the chain rule of probability:

$$p(w_1, \ldots, w_T) = \prod_{t=1}^{T} p(w_t \mid w_1, \ldots, w_{t-1})$$

So modelling sequences reduces to modelling the conditional $p(w_t \mid w_{1:t-1})$.

**Goal:** learn $f_\Theta : \mathcal{V}^* \to [0,1]^V$ that predicts the next token given all preceding tokens.

The **transformer** is the architecture for $f_\Theta$. It can attend to the *entire* preceding context.

# Token Embedding

Tokens are integers; we cannot differentiate with respect to them.

## Embedding Matrix $\mathbf{E} \in \mathbb{R}^{V \times d}$

The embedding of token $w_t$ is the $w_t$-th row of $\mathbf{E}$:

$$\mathbf{e}_t = \mathbf{E}[w_t] \in \mathbb{R}^{1 \times d}$$

Equivalently: one-hot $\mathbf{o}_t$ times $\mathbf{E}$ (a row lookup).

$\mathbf{E}$ is **learned**: tokens that play similar roles get similar embeddings. $d$ is the model dimension (e.g. 512, 4096, 12288).

# Positional Encoding

The embedding depends only on token identity, not position. But order matters: "dog bites man" $\neq$ "man bites dog".

**Fix:** add a position signal to each embedding:

$$\mathbf{x}_t = \mathbf{E}[w_t] + \mathbf{P}[t]$$

Vaswani et al. used deterministic sinusoids at different frequencies:

$$\mathbf{P}[t,\, 2k] = \sin\left(t/10000^{2k/d}\right)$$
$$\mathbf{P}[t,\, 2k{+}1] = \cos\left(t/10000^{2k/d}\right)$$

Each position gets a unique signature. The full input to block 1:

$$\mathbf{X} = \mathbf{X}_{\text{emb}} + \mathbf{P}_{1:N} \in \mathbb{R}^{N \times d}$$

# Motivation

*"The chicken didn't cross the road because* **it** *was too tired."*

To represent "it", we need to look at other tokens and decide which ones are relevant.

**Idea:** compute a weighted sum of all preceding representations:

$$\mathbf{a}_i = \sum_{j \le i} \alpha_{ij}\, \mathbf{x}_j, \qquad \alpha_{ij} \ge 0, \quad \sum_{j \le i} \alpha_{ij} = 1$$

The constraint $j \le i$ is the **causal mask**: we cannot look at future tokens (they don't exist yet at generation time).

# Queries, Keys, and Values

Using $\mathbf{x}_i$ for all three roles (query, key, value) is too restrictive. Separate them with learned projections:

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q \qquad \text{“what am I looking for?”}$$

$$\mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K \qquad \text{“what do I contain?”}$$

$$\mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V \qquad \text{“what info do I provide?”}$$

## Scaled Dot-Product Attention

$$\alpha_{ij} = \frac{\exp(\mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{d_k})}{\sum_{m \le i} \exp(\mathbf{q}_i \cdot \mathbf{k}_m / \sqrt{d_k})}, \qquad \mathbf{a}_i = \sum_{j \le i} \alpha_{ij} \mathbf{v}_j$$

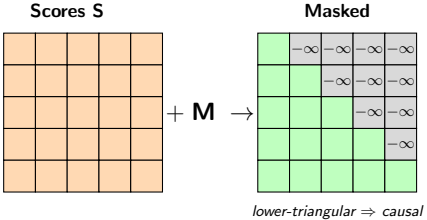
Dividing by $\sqrt{d_k}$ keeps score variance at $\sim 1$, preventing softmax saturation.

# The Causal Mask

In matrix form for all $N$ positions at once:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}\right)\mathbf{V}$$

where $M_{ij} = 0$ if $j \leq i$, and $-\infty$ otherwise.




Cost: $O(N^2 d_k)$ (quadratic in sequence length).

# Multiple Attention Heads

Tokens relate in many ways (syntax, coreference, semantics). A single head can only learn one pattern.

## Multi-Head Attention

Run $A$ independent heads, each with its own $\mathbf{W}^{Qc}, \mathbf{W}^{Kc}, \mathbf{W}^{Vc}$:

$$\text{MultiHead}(\mathbf{X}) = \left[\text{head}^1 \parallel \cdots \parallel \text{head}^A\right] \mathbf{W}^O$$

- Each head operates in a $d/A$-dimensional subspace
- Set $d_k = d_v = d/A$ so concatenation returns shape $[N \times d]$
- Original transformer: $d = 512$, $A = 8$, $d_k = 64$

# Residual Connections and LayerNorm

After each sub-layer (attention or FFN):

**Residual connection:**

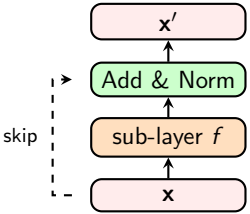
$$\mathbf{x}' = f(\mathbf{x}) + \mathbf{x}$$

Gradient is $\mathbf{I} + \frac{\partial f}{\partial \mathbf{x}}$: always at least the identity. Solves vanishing gradients.

**LayerNorm** (per token):

$$\mathrm{LN}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \mu}{\sigma + \epsilon} + \beta$$

Normalises each token's $d$-dim vector to zero mean, unit variance.



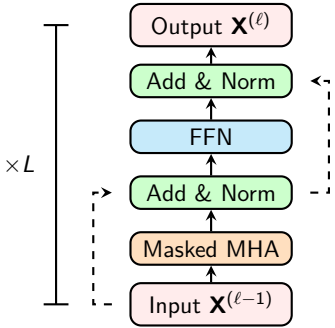
# Feed-Forward Network

A two-layer MLP applied **independently** to each position:

$$\mathrm{FFN}(\mathbf{x}) = \mathrm{ReLU}(\mathbf{x}\,\mathbf{W}_1 + \mathbf{b}_1)\,\mathbf{W}_2 + \mathbf{b}_2$$

- $\mathbf{W}_1 \in \mathbb{R}^{d \times d_{\mathrm{ff}}}$, $\mathbf{W}_2 \in \mathbb{R}^{d_{\mathrm{ff}} \times d}$ with $d_{\mathrm{ff}} = 4d$
- Expands to $4d$, applies nonlinearity, compresses back to $d$
- No interaction between positions (attention does that)

# One Block, Repeated $L$ Times




In pre-norm convention:

$$\mathbf{O}^{(\ell)} = \mathbf{X}^{(\ell-1)} + \mathrm{MHA}\big(\mathrm{LN}(\mathbf{X}^{(\ell-1)})\big)$$

$$\mathbf{X}^{(\ell)} = \mathbf{O}^{(\ell)} + \mathrm{FFN}\big(\mathrm{LN}(\mathbf{O}^{(\ell)})\big)$$

Input and output are both $[N \times d] \Rightarrow$ blocks stack.

**Params per block:** $12\,d^2$ ($4d^2$ attention + $8d^2$ FFN).

Total: $N_{\text{params}} \approx 12\,L\,d^2$.

GPT-3: $L = 96$, $d = 12288 \Rightarrow \sim$175B params.

# From Hidden State to Next Token

After $L$ blocks + final LayerNorm, take the last token's representation $\mathbf{h}_N \in \mathbb{R}^d$.

**Unembedding** (weight-tied with $\mathbf{E}$):

$$\mathbf{u} = \mathbf{h}_N \mathbf{E}^\top \in \mathbb{R}^V \qquad \text{(logits)}$$

**Softmax**:

$$p(w_{N+1} = k \mid w_{1:N}) = \frac{\exp(u_k)}{\sum_j \exp(u_j)}$$

Tokens whose embeddings are close to $\mathbf{h}_N$ get high probability. Weight tying halves the vocabulary parameters and acts as a regulariser.

# Training: Cross-Entropy Loss

At every position $t$, the model predicts $p(\cdot \mid w_{1:t})$. The loss is:

$$\mathcal{L} = -\frac{1}{T}\sum_{t=1}^{T} \log p_\Theta(w_{t+1} \mid w_{1:t})$$

- Minimised via Adam + backpropagation through all $L$ blocks
- Thanks to the causal mask, **all positions are computed in parallel** (unlike RNNs)
- This is the key advantage: massive parallelism during training

# Generation: Autoregressive Sampling

Given a prompt $w_{1:N}$, generate one token at a time:

1. Forward pass $\rightarrow$ logits $\rightarrow$ softmax $\rightarrow$ sample $w_{N+1}$
2. Append $w_{N+1}$ to the context, repeat
3. Stop at end-of-sequence token or max length

**Sampling strategies:**

- **Greedy:** $w_{t+1} = \arg\max_k p_k$ (deterministic, repetitive)
- **Top-$k$:** keep $k$ most probable, renormalise, sample
- **Top-$p$ (nucleus):** keep smallest set with cumulative prob $\geq p$, renormalise, sample

# Recap: The Complete Forward Pass

1. **Embed + Position:** $\mathbf{x}_t = \mathbf{E}[w_t] + \mathbf{P}[t]$
2. $L$ **Transformer Blocks**, each:
   - Masked multi-head attention (mix information across positions)
   - Add & LayerNorm
   - Feed-forward network (process each position independently)
   - Add & LayerNorm
3. **Final LayerNorm**
4. **Unembedding + Softmax:** $\mathbf{p} = \text{softmax}(\mathbf{h}_N \mathbf{E}^\top)$

## Key design principles

- Attention = the *only* cross-position operation
- Residuals + norms = stable gradient flow through depth
- Causal mask = parallel training, autoregressive generation

# Questions?